

Paper Replication Study: Scaling Laws for Neural Language Models

1 Introduction

Modern Natural Language Processing is driven by empirical scaling. In this capstone, you will transition from student to researcher by undertaking a rigorous replication study of the landmark OpenAI paper: **"Scaling Laws for Neural Language Models" (Kaplan et al., 2020)**.

Rather than completing a standard software engineering project, you will validate the core hypothesis of scaling theory: that language modeling cross-entropy loss follows predictable power-law relations with respect to model parameters (N) and dataset size (D).

This replication study is organized into two sequential levels:

- **Level 1: Architectural Implementation** – Constructing a clean, decoder-only autoregressive Transformer from foundational tensor operations in PyTorch.
- **Level 2: Scaling Law Validation** – Executing controlled training sweeps across parameter sizes and dataset scales to empirically fit and analyze the power-law exponents α_N and α_D .

Core Guidelines:

- **No High-Level Shortcuts:** Utilizing high-level PyTorch abstractions such as `torch.nn.Transformer` or `nn.MultiheadAttention` is strictly prohibited. All attention mechanics, tensor shaping, and masking projections must be coded from scratch.
 - **Strict Control of Variables:** To claim a valid replication, all optimization parameters, learning rate decay profiles, and token throughput limits must be meticulously controlled.
-

2 Level 1: Foundational Architecture (Decoder-Only GPT)

Your first objective is to build a mathematically precise autoregressive (decoder-only) Transformer.

2.1 Mathematical Formulations to Implement

You must implement the following components as modular PyTorch layers:

1. **Causal Multi-Head Attention (MHA):** Given an input tensor $X \in \mathbb{R}^{B \times T \times d_{\text{model}}}$, project it to Queries (Q), Keys (K), and Values (V) using linear projection weights. Split into H heads of dimension $d_k = d_{\text{model}}/H$ and compute attention using causal masking:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

where the causal mask $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{ij} = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases}$$

2. **Position-wise Feed-Forward Network (FFN):** A two-layer MLP mapping applied identically across sequence positions:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, with $d_{\text{ff}} = 4 \times d_{\text{model}}$.

3. **Pre-Layer Normalization (Pre-LN):** To ensure stable gradient flow when scaling layer depths, place Layer Normalization on the input branches:

$$x^{(k+1)} = x^{(k)} + \text{SubLayer}(\text{LN}(x^{(k)}))$$

4. **Positional Encodings:** Inject sequential order using fixed sinusoidal positional encodings or learnable position embeddings.
5. **Weight-Tied LM Head:** An output projection mapping back to vocabulary dimension V_{vocab} . To reduce parameter bloat, bind the weights of your token embedding matrix to the final linear layer.

2.2 Progression Checklist – Level 1

- ☐ Implement a custom data pipeline to pack raw tokens into batches of $X, Y \in \mathbb{R}^{B \times T}$.
- ☐ Build the causal attention head with an explicit lower-triangular mask.
- ☐ Write tensor dimension verification unit tests for each intermediate layer.
- ☐ Verify causal compliance: assert that gradients at sequence index t do not propagate to inputs at indices $> t$.
- ☐ Overfit a single batch of text to zero cross-entropy loss to verify mathematical completeness.

3 Level 2: Replicating Neural Scaling Laws

With a verified architecture, you will now replicate the empirical experiments of Kaplan et al. (2020). You will analyze scaling performance across two dimensions: parameters (N) and training token dataset size (D).

3.1 Parameter Counting Methodology

In order to focus on the expressive capacity of the core sequence model, embedding parameters are highly dependent on vocabulary size and must be **excluded** from your parameter count variable N . For a network with L layers, hidden dimension d_{model} , and $d_{\text{ff}} = 4d_{\text{model}}$, the non-embedding parameter count is defined as:

$$N \approx L \times (12 \cdot d_{\text{model}}^2 + 4 \cdot d_{\text{model}})$$

Implement the following programmatic validation in your training script:

```
def count_non_embedding_params(model):  
    return sum(p.numel() for name, p in model.named_parameters()  
               if 'embed' not in name and 'lm_head' not in name)
```

3.2 Replication Sweep Design

To preserve compute accessibility on free-tier platforms (Google Colab, Kaggle, Lightning AI), you will execute two distinct, low-overhead sweeps designed to complete in under 30 minutes:

1. **Parameter Scaling Sweep (Finding α_N):** Train three models of increasing dimensions (Table 1) on the full dataset for exactly 3,000 steps. Track the best validation loss (L) achieved by each model scale.

Table 1: Model Configurations for Parameter Sweep

Scale	Layers (L)	d_{model}	d_{ff}	Heads (H)	Context (T)	Approx. Parameters (N)
Tiny	2	64	256	2	128	$\approx 1.0 \times 10^5$
Small	4	128	512	4	256	$\approx 7.9 \times 10^5$
Medium	6	256	1024	8	256	$\approx 4.7 \times 10^6$

2. **Data Scaling Sweep (Finding α_D):** Keep the model size constant by utilizing the **Small** model configuration. Train this model on four subset sizes of your dataset: 10%, 25%, 50%, and 100% of total tokens. Record the minimal validation loss (L) for each data budget limit.

3.3 Dataset Options (No Custom Tokenizer Setup Required)

To eliminate the friction of building custom tokenizers, select one of the following ready-to-use datasets:

- **Tiny Shakespeare (Character-Level):** A 1.1MB character corpus. Use a simple 65-character Python dictionary lookup to map characters directly to integers. Training is extremely fast and yields stable curves.
- **TinyStories-10k (Subword-Level):** Synthetically generated, grammatically simple stories. Use the off-the-shelf pre-trained GPT-2 tokenizer directly via Hugging Face:

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

3.4 Fitting the Exponents

Fit power-law equations to your experimental data in log-log space:

$$\log(L(N)) = \text{constant} - \alpha_N \log(N)$$

$$\log(L(D)) = \text{constant} - \alpha_D \log(D)$$

Plot both lines of best fit to determine the empirical exponents α_N and α_D .

3.5 Progression Checklist – Level 2

- ☐ Prepare the chosen dataset and create a standard 90/10 training/validation split.
 - ☐ Instantiate Tiny, Small, and Medium models, printing validation counts for non-embedding parameters.
 - ☐ Run the parameter sweep, keeping optimizer hyperparameters, sequence lengths, and step counts identical.
 - ☐ Run the data sweep using the Small model across the four token-budget configurations.
 - ☐ Transform results into logarithmic space and use `numpy.polyfit` to extract the slopes (α_N and α_D).
 - ☐ Calculate the scaling ratio: $\gamma = \alpha_N / \alpha_D$.
 - ☐ Generate log-log scaling plots (`scaling_laws.png`) containing the regression lines and raw data points.
-

4 Analysis & Discussion

4.1 Comparing Exponents and the Scaling Ratio

Once you calculate your empirical parameters, analyze the scaling ratio:

$$\gamma = \frac{\alpha_N}{\alpha_D}$$

In Kaplan et al. (2020), OpenAI reported $\alpha_N \approx 0.076$ and $\alpha_D \approx 0.095$, implying $\gamma \approx 0.80$. A ratio $\gamma < 1.0$ suggests that scaling data yields faster marginal performance gains than scaling parameters on modern architectures. Contrast your empirical ratio with OpenAI's baseline and discuss what your observed ratio indicates regarding token efficiency vs. parameter capacity on your selected dataset.

5 Submission Guidelines

5.1 Repository Layout

Your repository must strictly follow this structure:

```
models.py          # Custom CausalAttention, FFN, and DecoderTransformer modules
train_scaling.py   # Main training script accepting sweep configurations
plot_scaling.py    # Exponent fitting and plot generation script
report.pdf         # Compiled research paper replication report
```

5.2 Research Report Requirements (Overleaf LaTeX)

Your replication report must read as a formal research paper. Document:

1. **Mathematical Formalism:** Define the exact formulas implemented in Level 1.
2. **Replication Methodology:** State dataset choices, token counts, and the optimization configuration.
3. **Scaling Curves:** Embed your log-log validation scaling plots. Include the empirical regression equations and calculated values for α_N , α_D , and the ratio γ .
4. **Theoretical Discussion:** Address the discrepancies with Kaplan et al. using the specific analysis criteria outlined above.

May your scaling curves be exponential, and your constants robust!